

Formation Data Science

Python, NumPy & Pandas — vers le métier de Data Analyst / ML (secteur assurance)

Contents

1	Feuille de route	2
2	NumPy	3
2.1	Opérations matricielles	3
2.2	Déterminant, inverse, trace, valeurs propres	3
2.3	Reshaping et manipulation de forme	4
2.4	Indexation avancée et masques booléens	4
2.5	Broadcasting	5
2.6	Génération de tableaux et nombres aléatoires	5
2.7	Statistiques avec NumPy	6
2.8	Vectorisation et fonctions universelles (ufuncs)	6
3	Pandas	7
3.1	A. Introduction et structures de données	7
3.2	B. Chargement et exploration de données	8
3.3	C. Sélection et filtrage	8
3.4	D. Nettoyage de données	8
3.5	E. Transformation et création de colonnes	9
3.6	F. Groupby et agrégations	9
3.7	G. Jointures et fusion de données	10
3.8	H. Statistiques descriptives	10
3.9	I. Export de données	11
3.10	Mini-projet — Analyse de risque et de sinistralité	11
4	Suite du programme	12
4.1	Visualisation (Matplotlib / Seaborn)	12
4.2	Statistiques appliquées et codées	12
4.3	Machine Learning — bases	12
4.4	Machine Learning — évaluation et optimisation	12
4.5	Projet pratique de bout en bout	12

1 Feuille de route

1. Python fondamentaux (**acquis**)
2. NumPy (**acquis**)
3. Pandas (**en cours**)
 - A. Introduction et structures de données (**acquis**)
 - B. Chargement et exploration de données (**acquis**)
 - C. Sélection et filtrage (**acquis**)
 - D. Nettoyage de données (**acquis**)
 - E. Transformation et création de colonnes (*à venir*)
 - F. Groupby et agrégations (*à venir*)
 - G. Jointures et fusion de données (*à venir*)
 - H. Statistiques descriptives (*à venir*)
 - I. Export de données (*à venir*)
4. Visualisation (Matplotlib / Seaborn)
5. Statistiques appliquées et codées
6. Data Cleaning & Preprocessing avancé
7. Machine Learning — bases (Scikit-learn)
8. Machine Learning — évaluation et optimisation
9. Deep Learning (optionnel)
10. Projet pratique de bout en bout

2 NumPy

2.1 Opérations matricielles

Définition : ensemble des opérations mathématiques appliquées aux matrices (tableaux 2D) : addition, multiplication élément par élément, produit matriciel, transposition.

Utilité : c'est le cœur du calcul en IA/ML — réseaux de neurones, régression, traitement d'images reposent tous sur des opérations matricielles rapides.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A + B # addition
A - B # soustraction
A * B # multiplication element par element
A @ B # produit matriciel (vrai produit)
np.dot(A, B) # equivalent a @
A * 3 # multiplication par un scalaire
A.T # transposition
np.linalg.matrix_power(A, 2) # A au carre
```

2.2 Déterminant, inverse, trace, valeurs propres

Déterminant — `np.linalg.det()`

Définition : indicateur numérique qui dit si une matrice est inversible.

Utilité : savoir si $\det(A) \neq 0$, donc si A est inversible.

```
np.linalg.det(A)
```

Inverse — `np.linalg.inv()`

Définition : matrice qui, multipliée par A, donne la matrice identité.

Utilité : résoudre des systèmes d'équations linéaires.

```
A_inv = np.linalg.inv(A)
A @ A_inv # doit donner la matrice identite
```

Trace — `np.trace()`

Définition : somme des éléments diagonaux d'une matrice.

Utilité : utile en statistiques (variance totale) et en algèbre linéaire.

```
np.trace(A)
```

Valeurs propres — `np.linalg.eig()`

Définition : valeurs et vecteurs associés qui caractérisent une transformation linéaire.

Utilité : base de la diagonalisation, de la PCA (réduction de dimension) en ML.

```
valeurs_propres, vecteurs_propres = np.linalg.eig(A)
```

2.3 Reshaping et manipulation de forme

Définition : changer la structure (dimensions) d'un tableau sans changer ses données.

Utilité : préparer des données avant de les donner à un modèle.

```
A = np.arange(12)
B = A.reshape(3, 4) # reorganise en 3 lignes, 4 colonnes
C = A.reshape(2, -1) # -1 : NumPy calcule la dimension manquante

D = B.flatten() # aplatit en 1D, renvoie une COPIE
E = B.ravel() # aplatit en 1D, renvoie une VUE si possible

F = B.T # transposition (inverse les axes)

G = np.array([1, 2, 3])
G2 = np.expand_dims(G, axis=0) # ajoute une dimension
G3 = np.squeeze(G2) # retire les dimensions de taille 1
```

2.4 Indexation avancée et masques booléens

Définition : sélectionner des éléments d'un tableau selon des conditions logiques ou des listes d'indices.

Utilité : filtrer rapidement des données sans boucle `for`.

```
A = np.array([1, -2, 3, -4, 5, -6])
A[A > 0] # indexation booléenne
A[A < 0] = 0 # modifier via un masque

B = np.array([1,2,3,4,5,6,7,8])
B[(B > 2) & (B < 7)] # ET logique
B[(B < 3) | (B > 6)] # OU logique

C = np.array([10,20,30,40,50])
C[[0, 2, 4]] # fancy indexing

M = np.array([[1,2,3],[4,5,6],[7,8,9]])
M[:, 1] # colonne 1
M[1, :] # ligne 1
M[0:2, 0:2] # sous-matrice

np.where(A > 0, A, 0) # condition avec choix de valeur
```

2.5 Broadcasting

Définition : mécanisme qui permet à NumPy d'effectuer des opérations entre tableaux de formes différentes en étirant virtuellement le plus petit.

Utilité : évite les boucles **for** pour appliquer une opération à chaque ligne/colonne.

```
M = np.array([[1,2,3],[4,5,6],[7,8,9]])
v = np.array([10, 20, 30])
M + v # v applique a chaque ligne

w = np.array([[100],[200],[300]])
M + w # w applique a chaque colonne

# Cas pratique : normaliser une matrice
data = np.array([[10,200,3],[20,400,6],[30,600,9]])
moyenne = data.mean(axis=0)
ecart_type = data.std(axis=0)
data_normalise = (data - moyenne) / ecart_type
```

Règle : deux dimensions sont compatibles si elles sont égales, ou si l'une des deux vaut 1 (comparaison de droite à gauche).

2.6 Génération de tableaux et nombres aléatoires

Fonction	Définition	Utilité
<code>np.random.rand()</code>	nombres aléatoires entre 0 et 1 (loi uniforme)	initialiser poids, données de test
<code>np.random.randn()</code>	nombres aléatoires loi normale (0,1)	simuler du bruit, des erreurs
<code>np.random.randint()</code>	entiers aléatoires dans un intervalle	données catégorielles, indices
<code>np.random.uniform()</code>	flottants aléatoires dans un intervalle	valeurs continues bornées
<code>np.random.choice()</code>	tirage aléatoire dans un tableau	échantillonnage, bootstrap
<code>np.random.seed()</code>	fixe la graine aléatoire	reproductibilité des résultats
<code>np.zeros()</code> / <code>np.ones()</code>	tableau rempli de 0 / de 1	initialisation
<code>np.full()</code>	tableau rempli d'une valeur donnée	initialisation avec valeur précise
<code>np.eye()</code>	matrice identité	algèbre linéaire
<code>np.arange()</code>	séquence de valeurs espacées régulièrement	générer des séquences/index
<code>np.linspace()</code>	N valeurs régulières entre deux bornes	discrétiser un intervalle

```
np.random.seed(42)
np.random.rand(2, 3)
np.random.randn(2, 3)
np.random.randint(0, 100, size=(3, 3))
np.random.uniform(5, 10, size=5)
np.random.choice([1,2,3,4,5], size=3, replace=False)
np.zeros((2, 3))
np.ones((2, 3))
np.full((2, 3), 7)
np.eye(3)
np.arange(0, 10, 2)
np.linspace(0, 1, 5)
```

2.7 Statistiques avec NumPy

```
np.mean([1,2,3,4]) # moyenne
np.median([1,2,3,100]) # mediane (robuste aux outliers)
np.std([1,2,3,4]) # ecart-type (dispersion)
np.var([1,2,3,4]) # variance
np.min(x); np.max(x) # extremes
np.percentile(x, 75) # percentile / quartile

M = np.array([[1,2,3],[4,5,6]])
np.mean(M, axis=0) # moyenne par colonne
np.mean(M, axis=1) # moyenne par ligne

np.corrcoef(x, y) # matrice de correlation
np.sum(x) # somme totale
np.cumsum(x) # somme cumulee
```

2.8 Vectorisation et fonctions universelles (ufuncs)

Définition : fonctions NumPy optimisées (écrites en C) qui s'appliquent élément par élément, sans boucle Python.

Utilité : remplacer les boucles `for` lentes par du calcul vectorisé, beaucoup plus rapide.

```
np.exp(x) # exponentielle (sigmoïde, softmax)
np.sqrt(x) # racine carree (distances, ecart-type)
np.log(x) # logarithme naturel
np.abs(x) # valeur absolue (erreurs, MAE)
np.round(x, 1)

np.where(x > 0, x, 0) # if/else vectorise

def carre_si_pair(x):
    return x**2 if x % 2 == 0 else x
f = np.vectorize(carre_si_pair)
f(np.array([1,2,3,4]))
```

3 Pandas

Support de cours : OpenClassrooms

Pandas est une bibliothèque construite au-dessus de NumPy, conçue pour manipuler des données **tabulaires** (lignes/colonnes), comme un fichier Excel ou une table de base de données. Elle permet de charger, explorer, nettoyer, filtrer, grouper et fusionner des données réelles — l'étape indispensable avant tout modèle de machine learning.

3.1 A. Introduction et structures de données

Series

Définition : colonne de données à une seule dimension, chaque valeur associée à une étiquette (index).

Utilité : représenter une seule variable (ex : liste d'âges de clients).

```
import pandas as pd

ages = pd.Series([25, 30, 35], index=['Alice', 'Bob', 'Chloe'])
```

DataFrame

Définition : tableau à deux dimensions (plusieurs colonnes/Series regroupées), avec noms de colonnes et index de lignes.

Utilité : structure centrale de Pandas pour tout dataset réel (nom, âge, prime, type de contrat...).

```
data = {'nom': ['Alice', 'Bob', 'Chloe'],
        'age': [25, 30, 35],
        'prime': [200, 350, 400]}
df = pd.DataFrame(data)
```

Attributs de base

```
df.shape # (lignes, colonnes)
df.columns # noms des colonnes
df.index # index des lignes
df.dtypes # type de chaque colonne
```

Explorer rapidement un DataFrame

Fonction	Définition	Utilité
<code>.head()</code> / <code>.tail()</code>	premières / dernières lignes	aperçu rapide sans surcharger
<code>.info()</code>	résumé technique (types, valeurs non nulles)	diagnostic, repérer les valeurs manquantes
<code>.describe()</code>	statistiques descriptives des colonnes numériques	vision statistique globale rapide

3.2 B. Chargement et exploration de données

```
df = pd.read_csv('donnees.csv')
df = pd.read_csv('donnees.csv', sep=';')
df = pd.read_csv('donnees.csv', nrows=100)

df = pd.read_excel('donnees.xlsx', sheet_name='Feuille1')
df = pd.read_json('donnees.json')

df.to_csv('resultat.csv', index=False) # export
```

Une fois chargé (quel que soit le format d'origine : CSV, Excel, JSON...), le fichier devient un DataFrame Pandas standard — toutes les fonctions Pandas s'appliquent ensuite de la même façon.

```
df['type_contrat'].unique() # valeurs distinctes
df['type_contrat'].nunique() # nombre de valeurs distinctes
df['type_contrat'].value_counts() # frequence de chaque valeur
```

3.3 C. Sélection et filtrage

```
df['age'] # une colonne (Series)
df[['nom', 'age']] # plusieurs colonnes (DataFrame)

df.loc[0] # ligne par LABEL
df.loc[0:2, ['nom', 'age']] # lignes/colonnes par label

df.iloc[0] # ligne par POSITION
df.iloc[0:2, 1] # lignes/colonnes par position

df[df['age'] > 30] # filtre boolean
df[(df['age'] > 25) & (df['prime'] > 300)] # conditions combinees
df[df['type_contrat'].isin(['auto', 'habitation'])] # appartenance a une liste
```

3.4 D. Nettoyage de données

```
df.isnull() # localise les valeurs manquantes
df.isnull().sum() # compte par colonne

df.dropna() # supprime lignes avec valeurs manquantes
df.dropna(subset=['age']) # seulement si 'age' est manquant
df.fillna(0) # remplace par une valeur fixe
df['age'].fillna(df['age'].mean()) # remplace par la moyenne

df.duplicated().sum() # compte les doublons
df.drop_duplicates() # supprime les doublons

df['age'] = df['age'].astype(int) # corriger un type
df['date_contrat'] = pd.to_datetime(df['date_contrat']) # convertir en date

df.rename(columns={'nom': 'nom_client'}, inplace=True) # renommer
```


3.5 E. Transformation et création de colonnes

Définition : créer de nouvelles colonnes à partir de colonnes existantes, ou modifier des colonnes en appliquant une fonction.

Utilité : construire des variables utiles pour l'analyse qui n'existent pas directement dans les données brutes.

```
# Créer une colonne simple
df['prime_annuelle'] = df['prime'] * 12

# Créer une colonne conditionnelle
df['profil_risque'] = np.where(df['age'] < 25, 'jeune', 'standard')

# apply() : appliquer une fonction personnalisée à une colonne
def categorie_age(age):
    if age < 25:
        return 'jeune'
    elif age < 60:
        return 'adulte'
    else:
        return 'senior'

df['categorie_age'] = df['age'].apply(categorie_age)

# apply() sur plusieurs colonnes à la fois (axis=1)
df['ratio'] = df.apply(lambda row: row['prime'] / row['age'], axis=1)

# map() : remplacer des valeurs selon un dictionnaire
df['type_contrat_code'] = df['type_contrat'].map({'auto': 1, 'habitation': 2})

# cut() : découper une variable numérique en tranches (bins)
df['tranche_age'] = pd.cut(df['age'], bins=[0, 25, 40, 60, 100],
                           labels=['18-25', '26-40', '41-60', '60+'])
```

3.6 F. Groupby et agrégations

Définition : regrouper les lignes d'un DataFrame selon une ou plusieurs colonnes, puis appliquer un calcul (moyenne, somme...) sur chaque groupe.

Utilité : l'outil numéro un pour répondre à des questions métier du type "en moyenne, par catégorie" — essentiel en assurance (prime moyenne par type de contrat, sinistres par région...).

```
# Groupby simple + une agregation
df.groupby('type_contrat')['prime'].mean()

# Plusieurs agregations a la fois
df.groupby('type_contrat')['prime'].agg(['mean', 'sum', 'count', 'std'])

# Grouper sur plusieurs colonnes
df.groupby(['type_contrat', 'tranche_age'])['prime'].mean()

# Agregations differentes par colonne
df.groupby('type_contrat').agg({
```

```

    'prime': 'mean',
    'nb_sinistres': 'sum',
    'age': 'max'
})

# size() : compter le nombre de lignes par groupe
df.groupby('type_contrat').size()

# reset_index() : remettre le resultat sous forme de DataFrame classique
resultat = df.groupby('type_contrat')['prime'].mean().reset_index()

```

3.7 G. Jointures et fusion de données

Définition : combiner plusieurs DataFrames en un seul, en les reliant par une colonne commune (comme une jointure SQL) ou en les empilant.

Utilité : les données réelles sont souvent réparties sur plusieurs tables (clients, contrats, sinistres) — il faut les assembler pour analyser.

```

clients = pd.DataFrame({'id_client': [1,2,3], 'nom': ['Alice','Bob','Chloe']})
contrats = pd.DataFrame({'id_client': [1,2,4], 'prime': [200,350,150]})

# merge() : jointure sur une colonne commune
pd.merge(clients, contrats, on='id_client', how='inner') # seulement les
correspondances
pd.merge(clients, contrats, on='id_client', how='left') # garde tout "clients"
pd.merge(clients, contrats, on='id_client', how='outer') # garde tout, meme sans
correspondance

# concat() : empiler des DataFrames (meme colonnes)
df_2023 = pd.DataFrame({'nom': ['Alice'], 'prime': [200]})
df_2024 = pd.DataFrame({'nom': ['Bob'], 'prime': [350]})
pd.concat([df_2023, df_2024], ignore_index=True)

# join() : equivalent a merge mais base sur l'index
clients.set_index('id_client').join(contrats.set_index('id_client'))

```

3.8 H. Statistiques descriptives

Définition : résumé chiffré des données d'un DataFrame ou d'une colonne : tendance centrale, dispersion, corrélations.

Utilité : étape d'analyse exploratoire avant de modéliser — comprendre la distribution et les relations entre variables.

```

df['prime'].mean()
df['prime'].median()
df['prime'].std()
df['prime'].var()
df['prime'].min(); df['prime'].max()
df['prime'].quantile(0.75)

# Correlation entre toutes les colonnes numeriques
df.corr()

```

```
# describe() par groupe
df.groupby('type_contrat')['prime'].describe()

# Tableau croise dynamique (pivot table)
df.pivot_table(values='prime', index='type_contrat',
                columns='tranche_age', aggfunc='mean')
```

3.9 I. Export de données

Définition : sauvegarder un DataFrame dans un fichier (CSV, Excel, JSON...) pour le partager ou le réutiliser.

Utilité : conserver le résultat de ton nettoyage/analyse sans avoir à tout refaire à chaque fois.

```
df.to_csv('resultat.csv', index=False)
df.to_excel('resultat.xlsx', index=False, sheet_name='Analyse')
df.to_json('resultat.json', orient='records')
```

3.10 Mini-projet — Analyse de risque et de sinistralité

Définition : projet de synthèse qui mobilise l'ensemble des notions Pandas vues ci-dessus, sur un portefeuille clients d'assurance auto fictif (`id_client`, `nom`, `age`, `type_contrat`, `date_contrat`, `prime`, `nb_sinistres`, `montant_sinistres`, `region`).

1. **Chargement :** `read_csv()`, `head()`, `info()`, `describe()`
2. **Nettoyage :** valeurs manquantes (`fillna`), doublons (`drop_duplicates`), types (`astype`, `to_datetime`)
3. **Transformation :** créer `tranche_age` (`cut`), `profil_risque` (`where/apply`)
4. **Analyse :** `groupby` — quelle tranche d'âge a le plus de sinistres ? quel type de contrat coûte le plus cher en moyenne ?
5. **Corrélations :** `df.corr()` entre `âge`, `prime`, `nb_sinistres`
6. **Conclusion :** synthèse des résultats, base pour un futur modèle ML de scoring de risque

4 Suite du programme

4.1 Visualisation (Matplotlib / Seaborn)

Graphiques, histogrammes, corrélations — essentiel pour explorer les données avant de modéliser.

4.2 Statistiques appliquées et codées

Moyenne, variance, corrélation, tests d'hypothèses, probabilités, distributions (normale, Poisson — utile pour la fréquence des sinistres en assurance).

4.3 Machine Learning — bases

Scikit-learn : régression linéaire/logistique, arbres de décision, forêts aléatoires. Cas d'usage assurance : prédire le risque, le montant d'un sinistre, détecter la fraude.

4.4 Machine Learning — évaluation et optimisation

Métriques (précision, rappel, RMSE), validation croisée, overfitting, tuning d'hyperparamètres.

4.5 Projet pratique de bout en bout

Un projet complet type “prédiction de sinistres” ou “scoring de risque client”, du nettoyage des données jusqu'au modèle.